

Sindh Madressatul Islam University Karachi

Object Oriented Programming



SPRING 2026

Lecture 7 of 16

Engr. Majid Kaleem

WEEKLY AGENDA

TENTATIVE WEEKLY DATES		TENTATIVE TOPICS
1	26 th -Jan – 30 th Jan	INTRO. TO OBJECT-ORIENTED PROGRAMMING, PROCEDURAL VS.OOP, OOP PRINCIPLES, ETC.
2	02 nd Feb – 06 th Feb	CLASSES & OBJECTS, JAVA NAMING CONVENTION, JAVA PACKAGES, & THE FOUR PILLARS OF OOP
3	09 th Feb – 13 th Feb	METHODS & MODULAR PROGRAMMING
4	16 th Feb – 20 th Feb	ABSTRACT CLASSES & INTERFACES ASSIGNMENT #1 & QUIZ #1
5	23 rd Feb – 27 th Feb	USE OF STATIC KEYWORD IN JAVA
6	02 nd Mar – 06 th Mar	POLYMORPHISM
7	09 th Mar – 13 th Mar	INTERFACES ASSIGNMENT #2 & QUIZ #2
8	16 th Mar – 20 th Mar	EXCEPTION HANDLING
← MID TERM EXAMINATIONS (23rd MAR 26 – 27th MAR 2026) →		
9	30 th Mar – 03 rd Apr	COLLECTIONS FRAMEWORK
10	06 th Apr – 10 th Apr	FILE HANDLING
11	13 th Apr – 17 th Apr	GUI PROGRAMMING
12	20 th Apr – 24 th Apr	APPLICATION DEVELOPMENT ASSIGNMENT #3 & QUIZ #3
13	27 th Apr – 01 st May	DESIGN PRINCIPLES & PATTERNS
14	04 th May – 08 th May	ADVANCED OOP CONCEPTS
15	11 th May – 15 th May	CASE STUDIES AND INDUSTRY PRACTICES ASSIGNMENT #4 & QUIZ #4
16	18 th May – 22 nd May	PRESENTATIONS & REVISION WEEK
← FINAL TERM EXAMINATIONS (25th MAY 26 – 29th MAY 2026) →		

Note

- This schedule is subject to revisions as conditions may warrant.*
- Topics will be covered in sequence no matter if the university observes any planned or unplanned holidays.*
- The information in this course outline is subject to revision as conditions may warrant.*

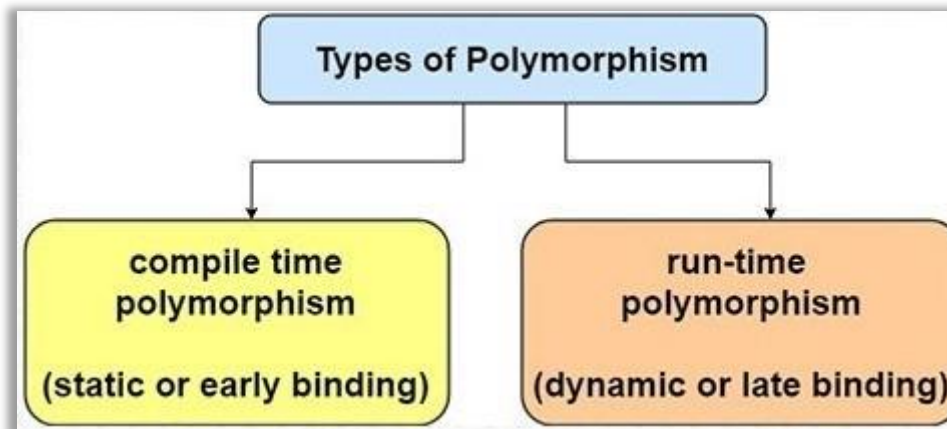
WEEK 07 - AGENDA

- Polymorphism in Java

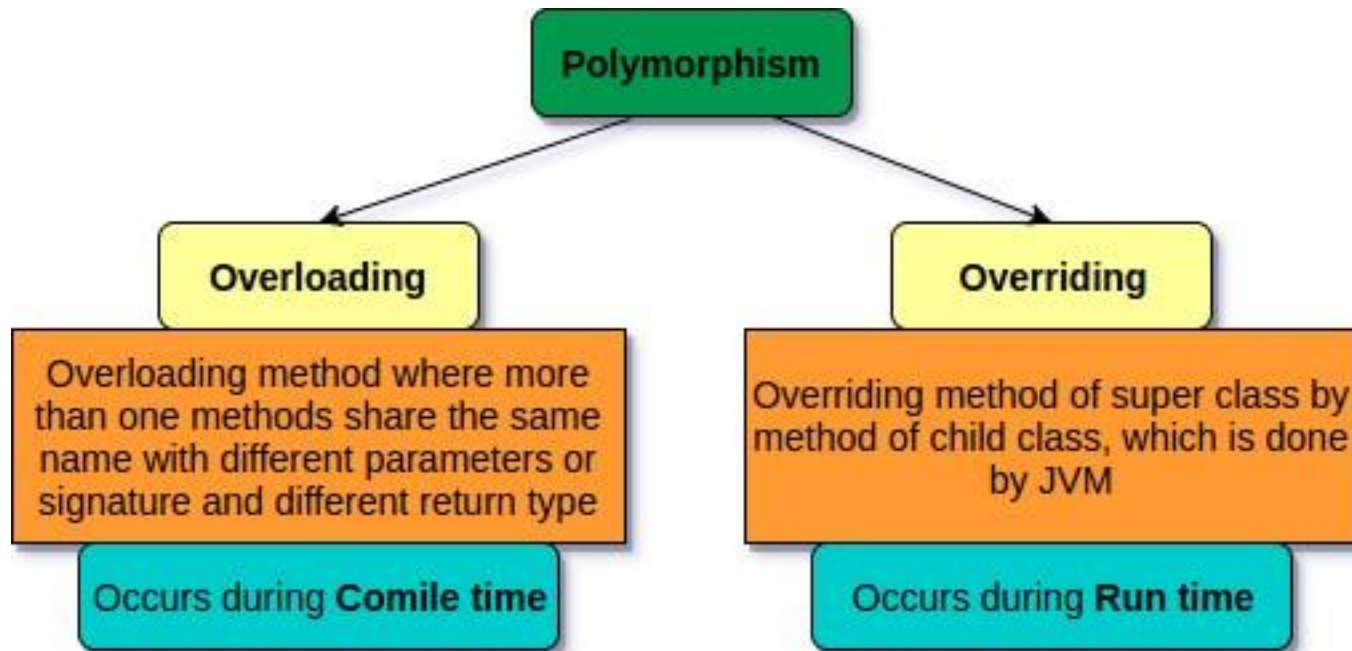
<https://onecompiler.com/java>

POLYMORPHISM

- **Real-life example of polymorphism in Java**
- Suppose if you are in class room that time you behave like a student, when you are in the market, then you behave like a customer, when you are at your home, then you behave like a son or daughter.
- Here, one person present in different-different behaviors.
- There are two types of polymorphism in Java:
 1. *Compile-time polymorphism (static binding)*
 2. *Runtime polymorphism (dynamic binding)*



POLYMORPHISM



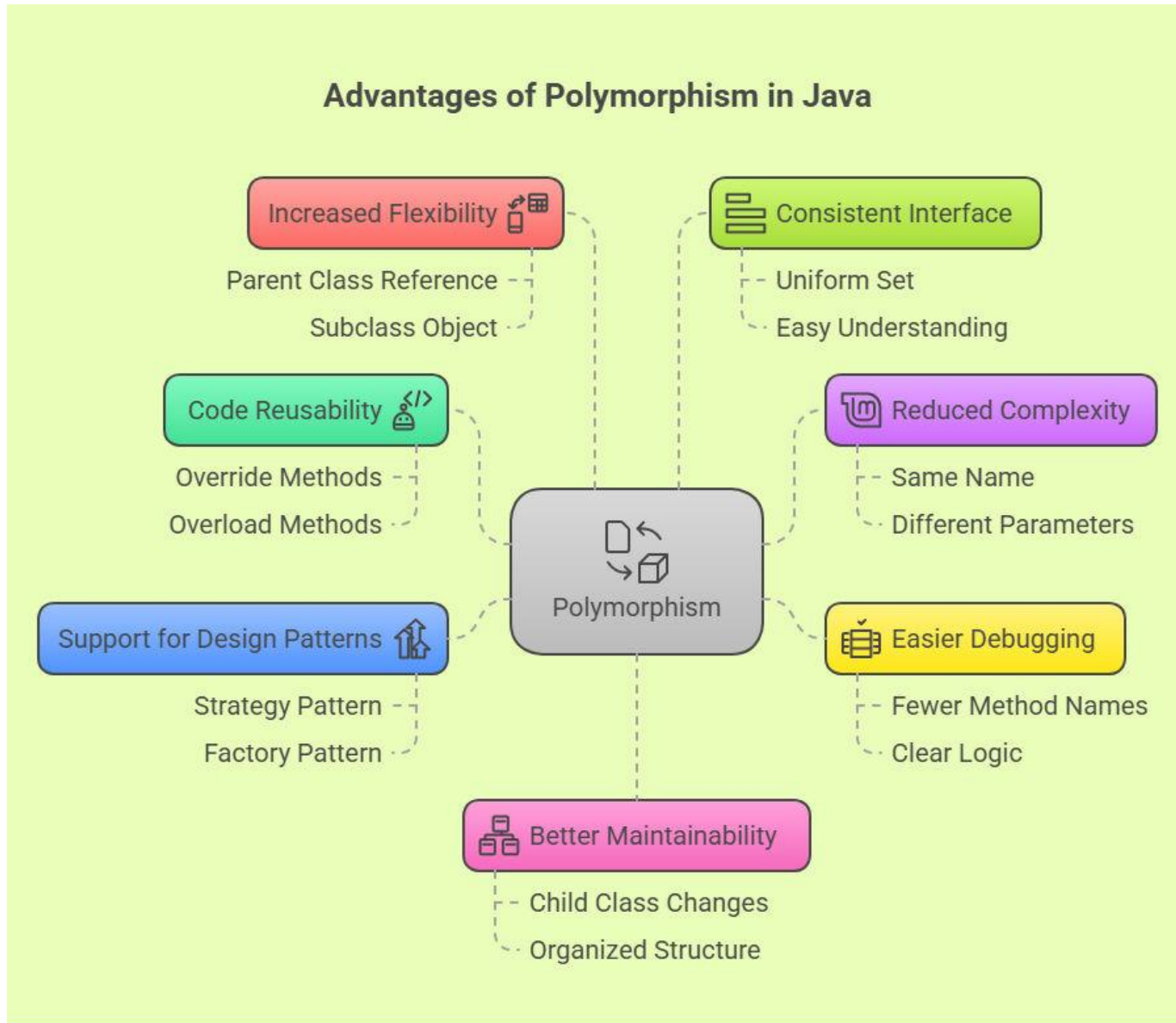
POLYMORPHISM

Aspect	Runtime Polymorphism in Java	Compile-time Polymorphism in Java
Alternate Name	Dynamic method dispatch	Static binding
Key Mechanism	Method overriding	Method overloading
Decision Time	Determined during program execution	Determined by the compiler before runtime
Inheritance Need	Requires a parent-child relationship	Can happen in the same class; no inheritance needed
Speed	Often a bit slower (dynamic checks)	Generally faster (compiler knows in advance)
Focus	Subclasses redefine a parent's method	Methods differ in parameters or argument types
Example Usage	Calling a shared method on various subclasses of a base class.	Having multiple versions of a method name within one class (e.g., different parameter lists).

POLYMORPHISM

- All major uses of Polymorphism in Java:
 1. Method Overloading (compile-time polymorphism)
 2. Method Overriding (runtime polymorphism)
 3. Interfaces
 4. Abstract Classes
 5. Upcasting
 6. Collections (real-world polymorphism)
 7. Anonymous Classes

ADVANTAGES OF POLYMORPHISM



ADVANTAGES OF POLYMORPHISM

- **Code Reusability:** You can keep a parent class's methods and simply override or overload them in different child classes. That approach spares you from rewriting large chunks of code.
- **Reduced Complexity:** Having multiple methods with the same name (but different parameters) saves you from introducing countless new names for minor variations of the same operation.
- **Easier Debugging:** When a handful of method names handle closely related tasks, you spend less time hunting through scattered functions. It's simpler to follow the logic and isolate bugs.
- **Support for Design Patterns:** Many standard patterns in Java, such as Strategy or Factory, rely on polymorphic behavior. They benefit from the flexibility of overriding and overloading.

ADVANTAGES OF POLYMORPHISM

- **Increased Flexibility:** A parent class reference can hold any subclass object. This makes your code far more dynamic because you can switch between subclasses without altering the main program flow.
- **Consistent Interface:** You maintain a uniform set of method names, so it's easier for you (and anyone on your team) to read and understand the purpose of each call.
- **Better Maintainability:** Once your classes and interfaces are defined, extending them with new functionality usually requires changing only the child classes. That keeps the overall structure organized.

1. Method Overloading (compile-time polymorphism)

– This happens when multiple methods have the same name but different parameter lists.

- Code:

2. Method Overriding (runtime polymorphism)

— This occurs when a subclass provides a specific implementation of a method already defined in its superclass.

- Code:

3. Polymorphism with Interfaces

- Interfaces allow multiple classes to implement the same contract differently.

- Code:

4. Polymorphism with Abstract Classes

- Abstract classes define methods that must be implemented by subclasses.

- Code:

5. Polymorphism with Upcasting

- Upcasting allows treating a subclass object as a superclass type.

- Code:

6. Polymorphism with Collections (real-world polymorphism)

- Java collections often store objects of different types but treat them uniformly.
- Code:

7. Polymorphism with Anonymous Classes

- An anonymous class in Java is essentially a class **without a name** that you **declare** and **instantiate** at the same time.
- It's defined **inline**, usually as part of an expression, and is used when you need a one-off implementation of an **interface** or **subclass** without formally creating a separate named class file.
- **Code:**

To be continued...

```
If(anyQuestions)
{
    askNow();
}
else
{
    thankYou();
    submitAttendance();
    endClass();
}
```